Hacker Women Council – Ethical Hacking

Actividad 03 - SQL & Web Security Challenge

Consultas SQL y Análisis de Bases de Datos

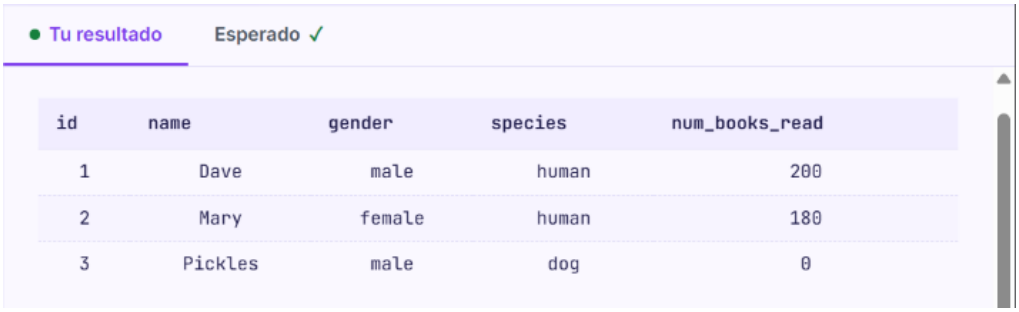
Alumno: Analía Rossotti

Fecha: Octubre de 2026

Ejercicio 1 – Seleccionar todo

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite seleccionar todos los registros y columnas de una tabla.

Consulta ejecutada: SELECT * FROM family_members;



Tu resultado Esperado ✓

id	name	gender	species	num_books_read
1	Dave	male	human	200
2	Mary	female	human	180
3	Pickles	male	dog	0

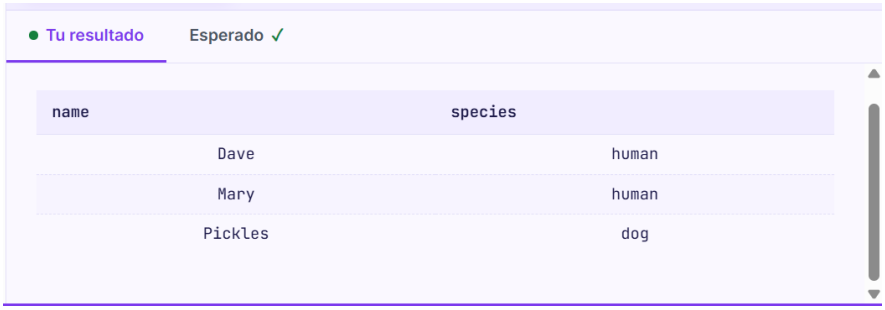
Resultado (Evidencia): El asterisco (*) actúa cómo -comodín- para indicar que devuelva todas las columnas disponibles (id, name, gender, species, num_books_read) de la tabla family_members.

Aprendí la estructura base para consultar una tabla completa sin filtrar campos específicos. Permite extraer el listado completo de una base de datos (por ejemplo, ver todos los usuarios registrados en una aplicación) para auditorías rápidas.

Ejercicio 2 – Selección de columnas específicas

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite proyectar únicamente columnas seleccionadas en lugar de toda la tabla, optimizando la visualización de datos.

Consulta ejecutada: SELECT name, species FROM family_members;



Tu resultado Esperado ✓

name	species
Dave	human
Mary	human
Pickles	dog

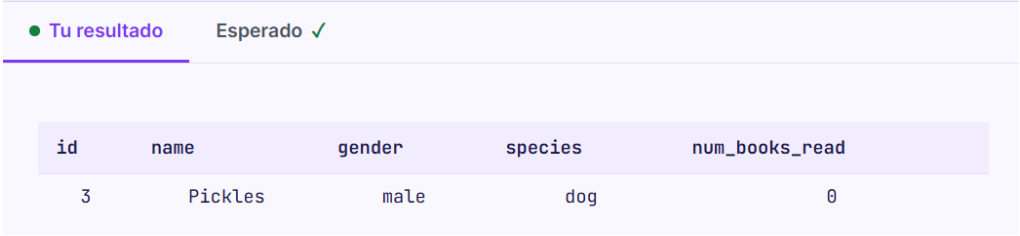
Resultado (Evidencia): La consulta devuelve exclusivamente las columnas name y species de la tabla family_members, omitiendo el resto de los campos.

Aprendí a filtrar columnas específicas utilizando SELECT seguida de los nombres de los campos separados por comas. Permite extraer información puntual de una base de datos (por ejemplo, obtener solamente los nombres y correos de los usuarios en una plataforma) sin exponer datos innecesarios.

Ejercicio 3 – Filtrado de registros con condición

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite filtrar filas específicas de una tabla utilizando la cláusula WHERE en función de una condición determinada.

Consulta ejecutada: SELECT * FROM family_members WHERE species = 'dog';



Resultado (Evidencia): La consulta devuelve únicamente el registro donde la especie es 'dog', mostrando los datos del miembro llamado Pickles (id 3, género male, 0 libros leídos).

Aprendí a utilizar la cláusula WHERE para restringir los resultados obtenidos y mostrar únicamente los datos que cumplen con un criterio específico. Permite buscar información puntual dentro de una base de datos grande para análisis precisos.

Ejercicio 4 – Filtrado con operador de comparación numérica

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite filtrar registros utilizando operadores de comparación numérica (>) junto con la cláusula WHERE.

Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read > 190;

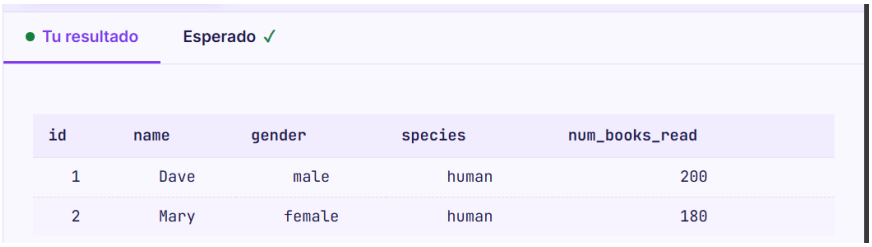


Resultado (Evidencia): La consulta devuelve únicamente el registro que cumple con la condición estricta de tener más de 190 libros leídos, mostrando los datos de Dave (id 1, especie human, 200 libros leídos).

Aprendí a utilizar operadores relacionales (>) en las consultas SQL para evaluar rangos numéricos. Permite filtrar datos cuantitativos de una base de datos (por ejemplo, buscar productos con precios mayores a cierto monto o usuarios con un puntaje específico) para análisis detallados.

Ejercicio 5 – Filtrado con operador de comparación mayor o igual que

Nivel de Riesgo Bajo (es una consulta de lectura básica). Permite filtrar registros utilizando el operador relacional de mayor o igual (>=) junto con la cláusula WHERE. **Consulta ejecutada:** SELECT * FROM family_members WHERE num_books_read >= 180;



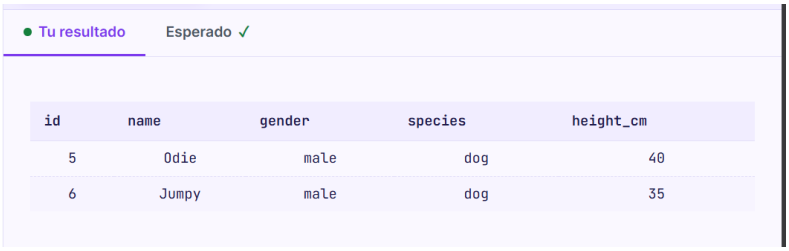
id	name	gender	species	num_books_read
1	Dave	male	human	200
2	Mary	female	human	180

Resultado (Evidencia): La consulta devuelve todos los registros que cumplen con la condición de tener un número de libros leídos mayor o igual a 180, mostrando los datos de Dave (id 1, 200 libros) y Mary (id 2, 180 libros).

Aprendí a utilizar el operador >= para incluir tanto los valores superiores como el límite exacto especificado en el filtro. Permite realizar búsquedas inclusivas de rangos numéricos en bases de datos (por ejemplo, filtrar empleados con una antigüedad mayor o igual a ciertos años o productos con un precio base determinado) para obtener análisis más completos.

Ejercicio 6 – Filtrado avanzado con operador lógico AND

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite combinar múltiples condiciones de filtrado simultáneas utilizando el operador lógico AND en la cláusula WHERE. **Consulta ejecutada:** SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';



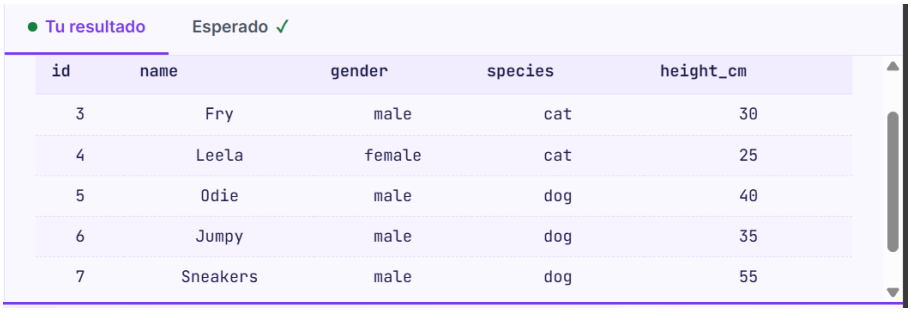
id	name	gender	species	height_cm
5	Odie	male	dog	40
6	Jumpy	male	dog	35

Resultado (Evidencia): La consulta devuelve los registros que cumplen estrictamente con ambas condiciones al mismo tiempo (tener una altura menor a 45 cm y pertenecer a la especie 'dog'), mostrando los datos de Odie (id 5, 40 cm) y Jumpy (id 6, 35 cm).

Aprendí a utilizar el operador AND para acotar los resultados cruzando dos o más criterios de búsqueda en una base de datos. Permite realizar consultas avanzadas y específicas (por ejemplo, filtrar productos que estén en oferta y además pertenezcan a una categoría concreta) para optimizar la obtención de información.

Ejercicio 7 – Filtrado flexible con operador lógico OR

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite buscar registros que cumplan con al menos una de múltiples condiciones utilizando el operador lógico OR en la cláusula WHERE. **Consulta ejecutada:** SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';



● Tu resultado Esperado ✓

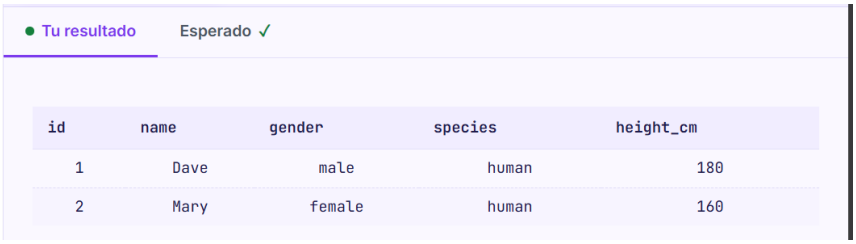
id	name	gender	species	height_cm
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

Resultado (Evidencia): La consulta devuelve todos los registros que coinciden con cualquiera de los criterios establecidos (medir menos de 45 cm de altura o pertenecer a la especie 'dog'), mostrando los datos de Fry, Leela, Odie, Jumpy y Sneakers.

Aprendí a utilizar el operador OR para ampliar los resultados obtenidos al evaluar condiciones alternativas. Permite realizar búsquedas abiertas en bases de datos (por ejemplo, buscar usuarios que pertenezcan a una ciudad específica o que se hayan registrado en una fecha determinada) para obtener conjuntos de datos más inclusivos.

Ejercicio 8 – Filtrado con lista de valores utilizando IN

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite buscar filas donde un valor coincida con elementos dentro de una lista específica utilizando el operador IN junto con la cláusula WHERE. **Consulta ejecutada:** SELECT * FROM friends_of_pickles WHERE species IN ('human');



● Tu resultado Esperado ✓

id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160

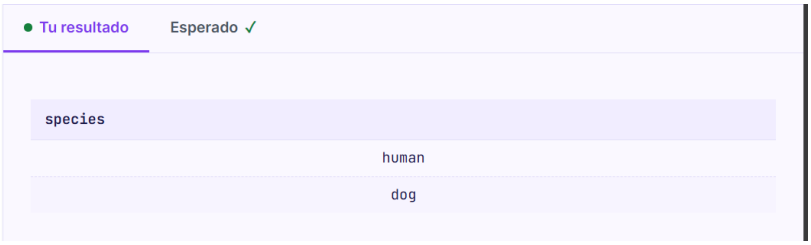
Resultado (Evidencia): La consulta devuelve los registros cuya especie coincide con el valor indicado en la lista ('human'), mostrando los datos de Dave (id 1) y Mary (id 2).

Aprendí a utilizar el operador IN para filtrar de manera limpia y eficiente valores específicos dentro de una columna. Permite realizar búsquedas estructuradas en bases de datos (por ejemplo, filtrar registros que pertenezcan a un conjunto cerrado de categorías o estados) para obtener reportes precisos.

Ejercicio 9 – Selección de valores únicos con DISTINCT y filtrado

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite combinar la eliminación de registros duplicados (DISTINCT) con una condición de exclusión (WHERE !=) para filtrar resultados específicos.

Consulta ejecutada: SELECT DISTINCT species FROM friends_of_pickles WHERE species != 'cat';



Resultado (Evidencia): La consulta devuelve únicamente las especies distintas que no corresponden a la categoría 'cat', mostrando exclusivamente las filas esperadas (human y dog).

Aprendí a combinar la palabra clave DISTINCT junto con operadores de desigualdad (!=) para limpiar resultados repetidos y excluir categorías no deseadas simultáneamente. Permite depurar bases de datos y obtener listados únicos restringidos (por ejemplo, ver los roles de usuario únicos en un sistema excluyendo a los administradores).

Ejercicio 10 – Ordenamiento descendente de resultados con ORDER BY y DESC

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite ordenar las filas devueltas de una consulta en sentido descendente basándose en un atributo específico utilizando la cláusula ORDER BY acompañada de DESC. **Consulta ejecutada:** SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;



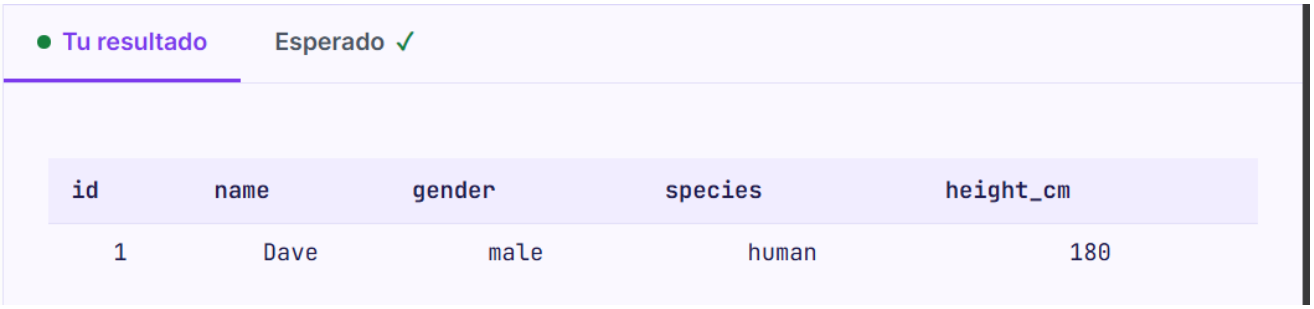
id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160
7	Sneakers	male	dog	55
5	Odie	male	dog	40
6	Jumpy	male	dog	35
3	Fry	male	cat	30
4	Leela	female	cat	25

Resultado (Evidencia): La consulta devuelve todos los registros de la tabla ordenados de mayor a menor altura (height_cm), mostrando primero a los individuos más altos (como Dave con 180 cm y Mary con 160 cm) y descendiendo hasta los de menor estatura.

Aprendí a utilizar la cláusula ORDER BY junto con la palabra clave DESC para estructurar y jerarquizar grandes volúmenes de datos según criterios numéricos o alfabéticos inversos. Permite organizar reportes y listados (por ejemplo, ordenar productos de mayor a menor precio o visualizar las transacciones más recientes o costosas primero) para análisis ágiles.

Ejercicio 11 – Limitación de resultados con LIMIT y ORDER BY

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite restringir la cantidad de filas devueltas por una consulta utilizando la cláusula LIMIT, frecuentemente combinada con ORDER BY para obtener los registros superiores o inferiores de un conjunto ordenado. **Consulta ejecutada:** SELECT * FROM friends_of_pickles WHERE name = 'Dave' ORDER BY height_cm LIMIT 1;

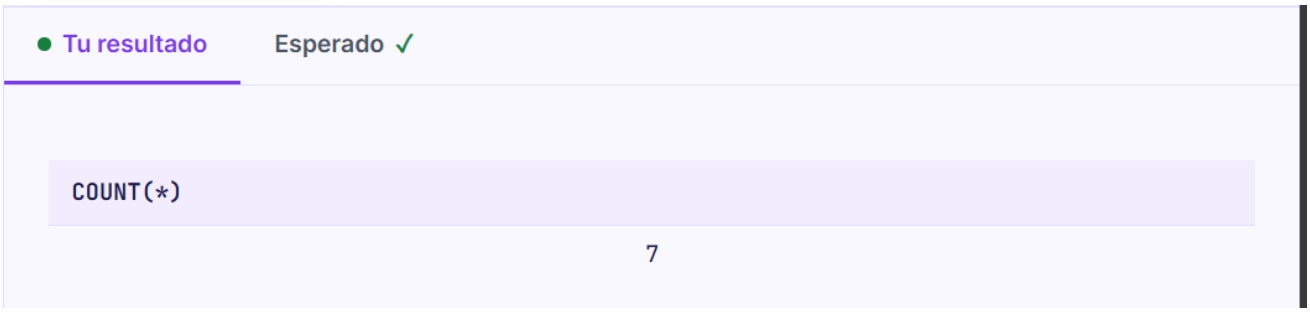


Resultado (Evidencia): La consulta devuelve exclusivamente la primera fila correspondiente al registro que cumple con la condición del nombre y el ordenamiento especificado, mostrando únicamente los datos de Dave (id 1, height_cm 180).

Aprendí a utilizar la cláusula LIMIT para controlar el volumen máximo de filas devueltas en una consulta SQL. Permite optimizar el rendimiento al consultar tablas masivas y extraer registros específicos (por ejemplo, obtener únicamente el producto más caro, el cliente con mayor puntaje o los primeros registros de una auditoría) para reportes directos.

Ejercicio 12 – Conteo total de filas con la función COUNT

Nivel de Riesgo Bajo (es una consulta de lectura básica). Permite calcular y verificar el número total de registros contenidos en una tabla o conjunto de resultados utilizando la función agregada COUNT(*). **Consulta ejecutada:** SELECT COUNT(*) FROM friends_of_pickles;

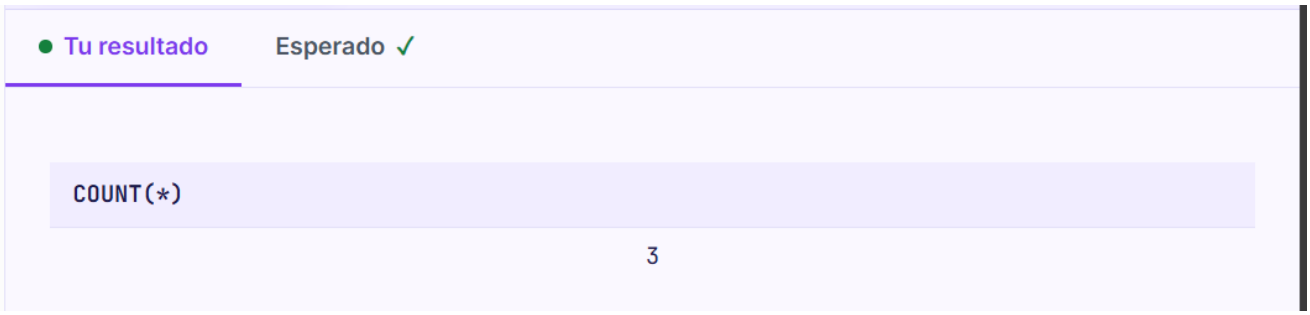


Resultado (Evidencia): La consulta devuelve el valor numérico total de filas de la tabla analizada, mostrando un resultado de 7 registros totales en friends_of_pickles.

Aprendí a utilizar la función de agregación COUNT(*) para realizar conteos rápidos de registros en bases de datos. Permite obtener métricas generales y estadísticas básicas (por ejemplo, contar el número total de clientes registrados, productos activos o transacciones realizadas) para auditorías y resúmenes de información.

Ejercicio 13 – Conteo condicional de filas con COUNT y WHERE

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite combinar la función de agregación COUNT(*) con la cláusula WHERE para contar únicamente los registros que cumplen con una condición específica**. Consulta ejecutada:** SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';

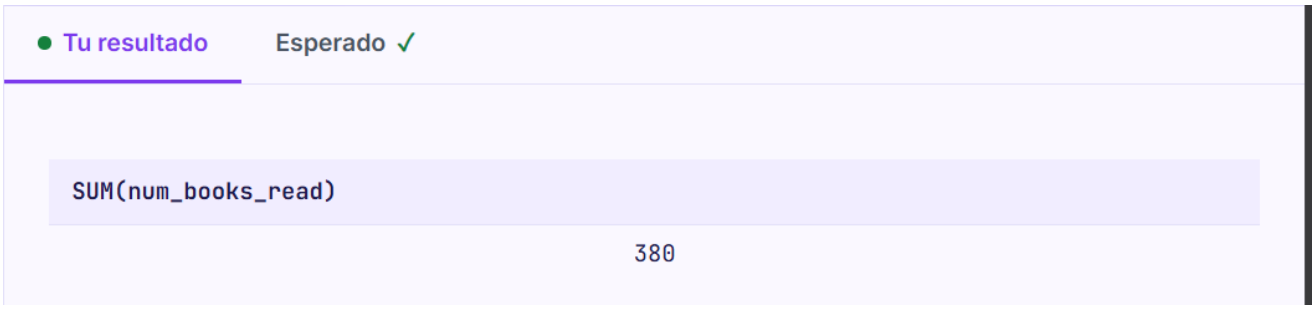


Resultado (Evidencia): La consulta devuelve el número total de registros que coinciden con la condición de pertenecer a la especie 'dog', mostrando un resultado de 3 elementos que cumplen con dicho criterio.

Aprendí a utilizar la función COUNT(*) en conjunto con filtros condicionales WHERE para obtener métricas segmentadas en lugar de un conteo global. Permite realizar análisis estadísticos precisos y específicos (por ejemplo, contar cuántos usuarios están activos en una región, cuántos productos tienen un stock determinado o cuántas transacciones superan un monto) para reportes detallados.

Ejercicio 14 – Sumatoria de valores numéricos con SUM

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite calcular la suma total de los valores contenidos en una columna numérica utilizando la función agregada SUM. **Consulta ejecutada:** SELECT SUM(num_books_read) FROM family_members;

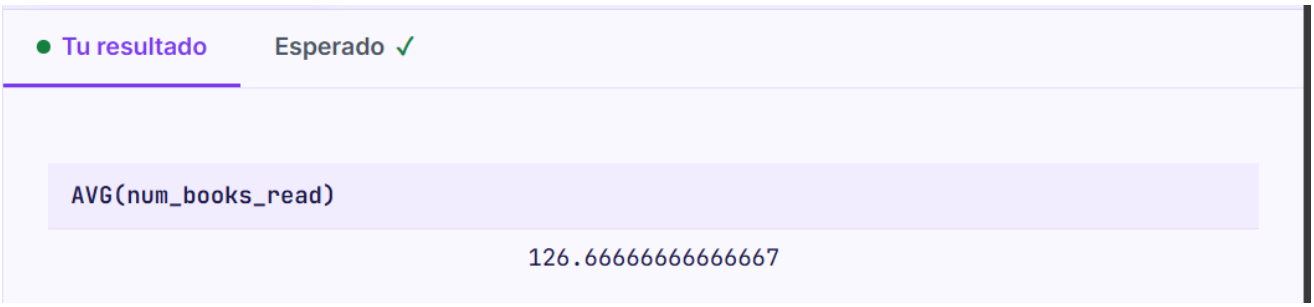


Resultado (Evidencia): La consulta devuelve el valor numérico acumulado de toda la columna especificada, mostrando un resultado total de 380 libros leídos entre los registros de la tabla family_members.

Aprendí a utilizar la función de agregación SUM para realizar operaciones aritméticas y cálculos acumulativos sobre conjuntos de datos cuantitativos. Permite obtener métricas globales importantes (por ejemplo, calcular el total de ventas mensuales, la suma de stock disponible de un inventario o el monto total facturado) para análisis financieros y reportes gerenciales.

Ejercicio 15 – Cálculo del promedio aritmético con AVG

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite calcular el valor promedio (la media aritmética) de los datos contenidos en una columna numérica utilizando la función agregada AVG. **Consulta ejecutada:** SELECT AVG(num_books_read) FROM family_members;

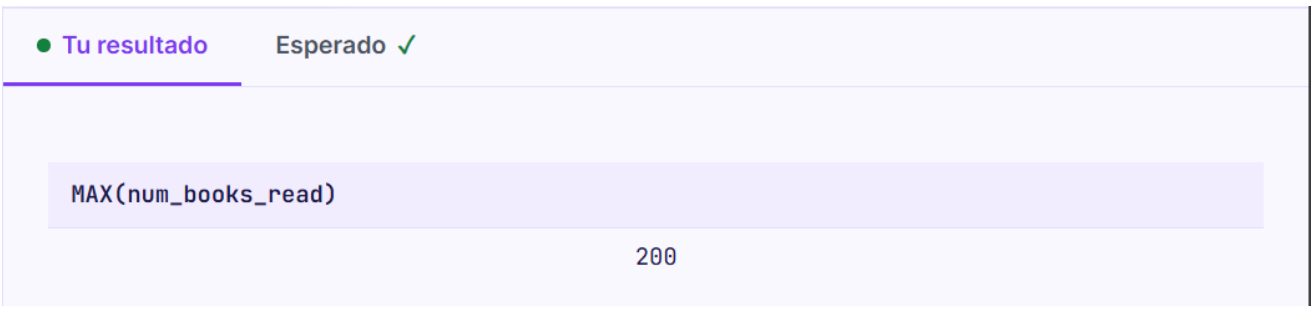


Resultado (Evidencia): La consulta devuelve el valor promedio calculado a partir de los datos de la columna especificada, mostrando un resultado de 126.666666666667 como la media de libros leídos por los miembros de la familia.

Aprendí a utilizar la función de agregación AVG para realizar análisis estadísticos de tendencia central sobre conjuntos de datos numéricos. Permite obtener métricas clave (por ejemplo, calcular el promedio de ventas mensuales, el precio medio de los productos en un inventario o la calificación promedio de los usuarios) para reportes analíticos.

Ejercicio 16 – Obtención del valor máximo con MAX

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite encontrar el valor numérico más alto dentro de una columna específica utilizando la función agregada MAX. **Consulta ejecutada:** SELECT MAX(num_books_read) FROM family_members;

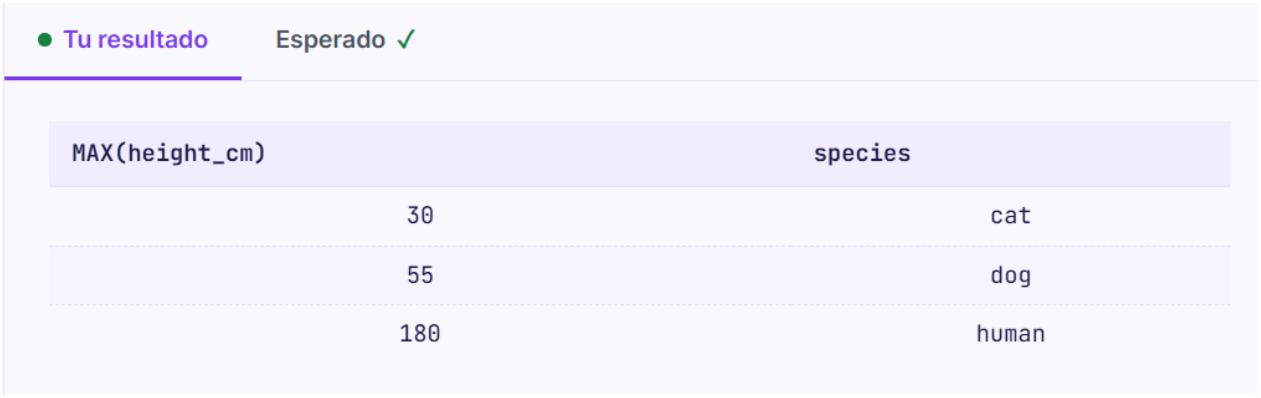


Resultado (Evidencia): La consulta devuelve el valor máximo contenido en la columna seleccionada, mostrando un resultado de 200 como la cantidad más alta de libros leídos por un miembro en la tabla family_members.

Aprendí a utilizar la función de agregación MAX para identificar límites superiores y extremos en conjuntos de datos cuantitativos. Permite extraer información crítica (por ejemplo, conocer el precio máximo de un producto, la temperatura más alta registrada o la venta más costosa) para análisis puntuales de rendimiento o auditorías.

Ejercicio 17 – Agrupamiento y valor máximo con GROUP BY y MAX

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite agrupar los registros por una columna específica y calcular el valor máximo asociado para cada grupo utilizando la función MAX combinada con la cláusula GROUP BY. **Consulta ejecutada:** SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;

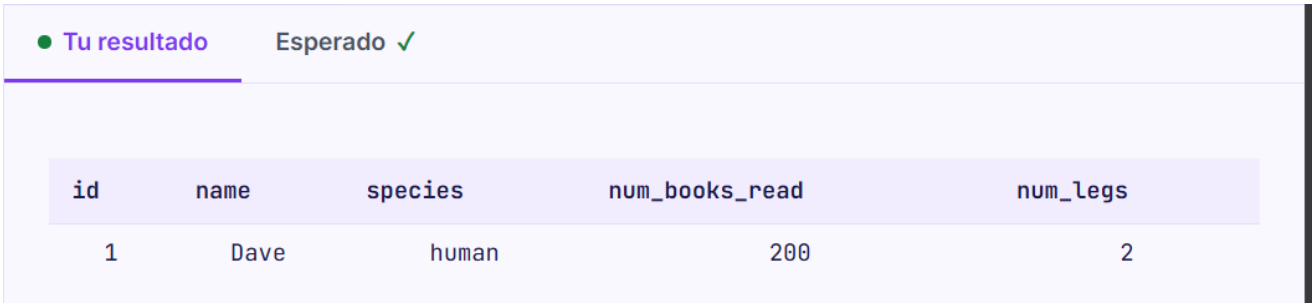


Resultado (Evidencia): La consulta devuelve el valor máximo de altura (MAX(height_cm)) junto con el nombre de cada especie (species), mostrando métricas independientes agrupadas correctamente para los gatos (30), perros (55) y humanos (180).

Aprendí a combinar la agrupación de datos (GROUP BY) con funciones de agregación como MAX para obtener estadísticas segmentadas por categorías. Permite realizar análisis avanzados y reportes detallados por categorías en bases de datos (por ejemplo, conocer el precio máximo de un producto en cada categoría o la venta más alta por región) de forma eficiente.

Ejercicio 18 – Consultas anidadas (Subconsultas / Subqueries)

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite utilizar el resultado de una consulta interna (subconsulta) como condición de filtro dentro de una consulta externa mediante la cláusula WHERE. **Consulta ejecutada:** SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);

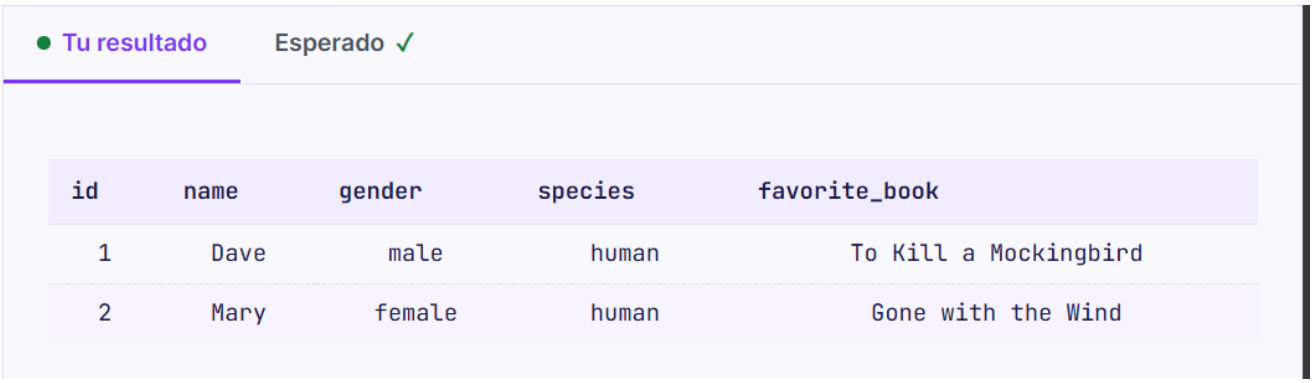


Resultado (Evidencia): La subconsulta SELECT MAX(num_books_read) FROM family_members devuelve primero el valor máximo (200), y la consulta principal utiliza dicho resultado para traer todos los datos completos del miembro correspondiente (Dave, id 1).

Aprendí a implementar subconsultas o consultas anidadas para dinaminzar los criterios de búsqueda sin necesidad de hardcodear valores estáticos. Permite realizar análisis avanzados en bases de datos (por ejemplo, buscar todos los empleados que ganan más que el salario promedio o identificar las transacciones que igualan el monto máximo registrado) de manera dinámica.

Ejercicio 19 – Filtrado de valores nulos con IS NOT NULL

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite filtrar y excluir aquellos registros donde una columna específica carece de valor (NULL) utilizando los operadores condicionales IS NULL o IS NOT NULL**. Consulta ejecutada:** SELECT * FROM family_members WHERE favorite_book IS NOT NULL;



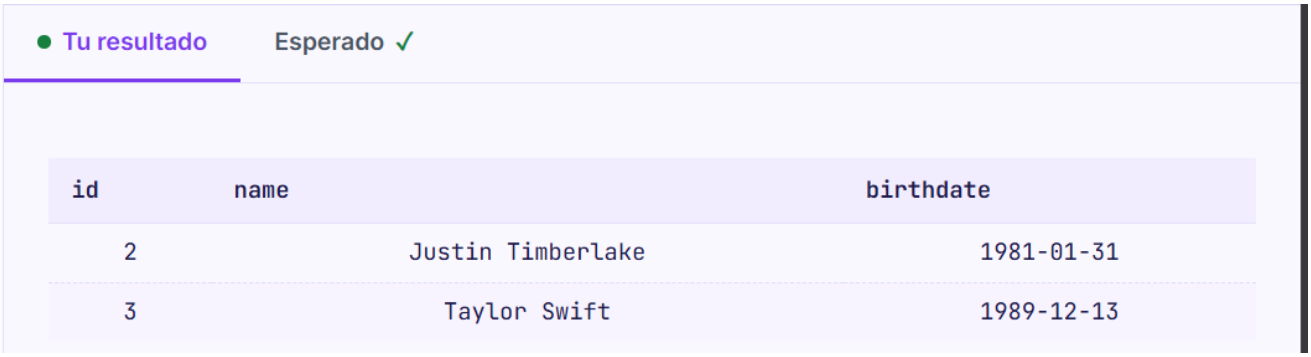
id	name	gender	species	favorite_book
1	Dave	male	human	To Kill a Mockingbird
2	Mary	female	human	Gone with the Wind

Resultado (Evidencia): La consulta devuelve exclusivamente las filas de la tabla family_members que tienen un libro favorito asignado, omitiendo el registro de Pickles que contiene un valor nulo (NULL) en dicha columna.

Aprendí a utilizar los operadores lógicos especiales IS NULL y IS NOT NULL para manejar adecuadamente la ausencia de datos en las bases de datos. Permite realizar filtrados limpios en auditorías de integridad (por ejemplo, buscar cuentas de usuario que sí tengan un correo o teléfono registrado, o detectar registros incompletos en bases de datos relacionales).

Ejercicio 20 – Comparación de fechas con operadores relacionales

Nivel de Riesgo **Bajo** (es una consulta de lectura básica). Permite filtrar registros basándose en campos de tipo fecha (DATE) utilizando operadores de comparación relacionales como mayor que (>) o menor que (<). **Consulta ejecutada:** SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';



id	name	birthdate
2	Justin Timberlake	1981-01-31
3	Taylor Swift	1989-12-13

Resultado (Evidencia): La consulta devuelve las celebridades cuya fecha de nacimiento es posterior al 1 de septiembre de 1980, mostrando correctamente las filas correspondientes a Justin Timberlake (1981-01-31) y Taylor Swift (1989-12-13).

Aprendí a utilizar operadores de comparación relacionales para evaluar datos de tipo fecha (DATE) expresados en formato estándar ISO (YYYY-MM-DD). Permite realizar filtrados temporales en auditorías (por ejemplo, consultar eventos ocurridos después de una fecha límite, transacciones recientes o registros de acceso creados a partir de un período específico).

Ejercicio 21 – Relación de tablas con INNER JOIN

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite combinar registros de dos o más tablas diferentes relacionándolas mediante una columna común utilizando la cláusula JOIN (o INNER JOIN) junto con la condición ON. **Consulta ejecutada:** SELECT character.name, character_actor.actor_name FROM character JOIN character_actor ON character.id = character_actor.character_id;

● Tu resultado Esperado ✓

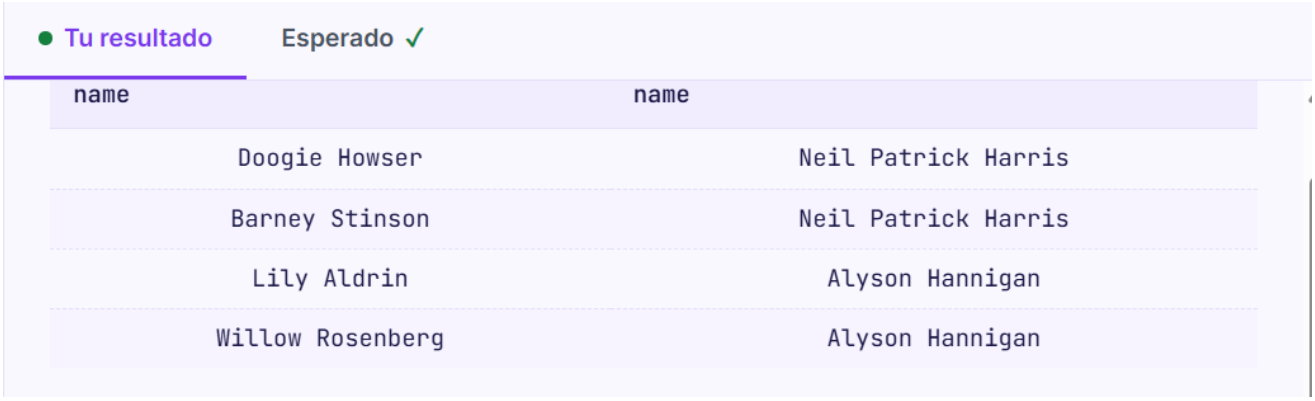
name	actor_name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan

Resultado (Evidencia): La consulta cruza con éxito la información de la tabla character con la tabla character_actor, devolviendo un listado unificado que asocia correctamente el nombre de cada personaje con el actor correspondiente (por ejemplo, relacionando a Doogie Howser y Barney Stinson con Neil Patrick Harris).

Aprendí a utilizar la cláusula JOIN para interconectar tablas relacionales mediante llaves primarias y foráneas. Permite estructurar consultas complejas para extraer y unificar información distribuida en múltiples tablas (por ejemplo, asociar clientes con sus respectivas compras, empleados con sus departamentos o usuarios con sus roles).

Ejercicio 22 – Consultas multitabla con múltiples JOIN (Múltiples joins)

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite conectar tres o más tablas utilizando múltiples cláusulas INNER JOIN secuenciales a través de tablas intermedias de relación para asociar entidades complejas. **Consulta ejecutada:** SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;

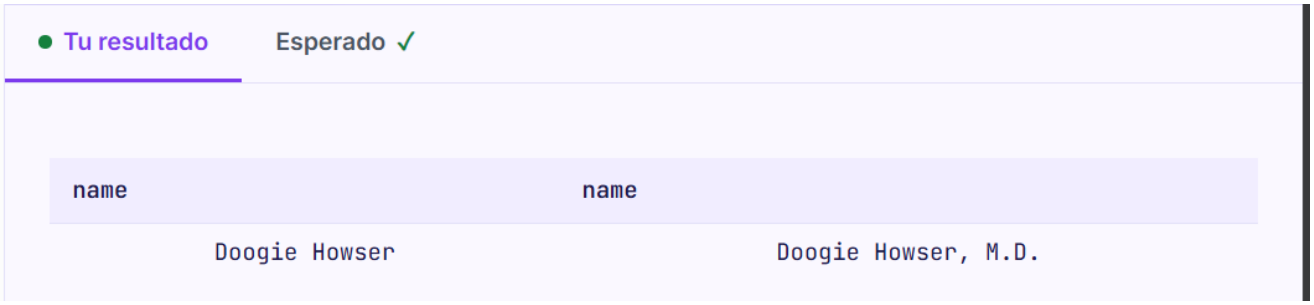


name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan

Resultado (Evidencia): La consulta enlaza de manera exitosa la tabla principal character con la tabla intermedia character_actor y finalmente con la tabla actor, devolviendo un listado integrado que asocia los nombres de los personajes con sus actores correspondientes sin duplicaciones innecesarias. Aprendí a encadenar múltiples uniones (JOIN) sucesivas para resolver esquemas relacionales complejos que involucran tablas intermedias de muchos a muchos. Permite estructurar consultas avanzadas en bases de datos relacionales (por ejemplo, relacionar clientes, pedidos y productos, o usuarios, roles y permisos en una auditoría de seguridad) extrayendo información estructurada de múltiples fuentes.

Ejercicio 23 – Combinación de JOIN y filtrado condicional con WHERE

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite aplicar filtros de exclusión y condiciones específicas sobre resultados obtenidos mediante múltiples uniones de tablas utilizando las cláusulas JOIN combinadas con WHERE y operadores relacionales de desigualdad (<>). **Consulta ejecutada:** SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name <> 'Willow Rosenberg' AND tv_show.name <> 'How I Met Your Mother';

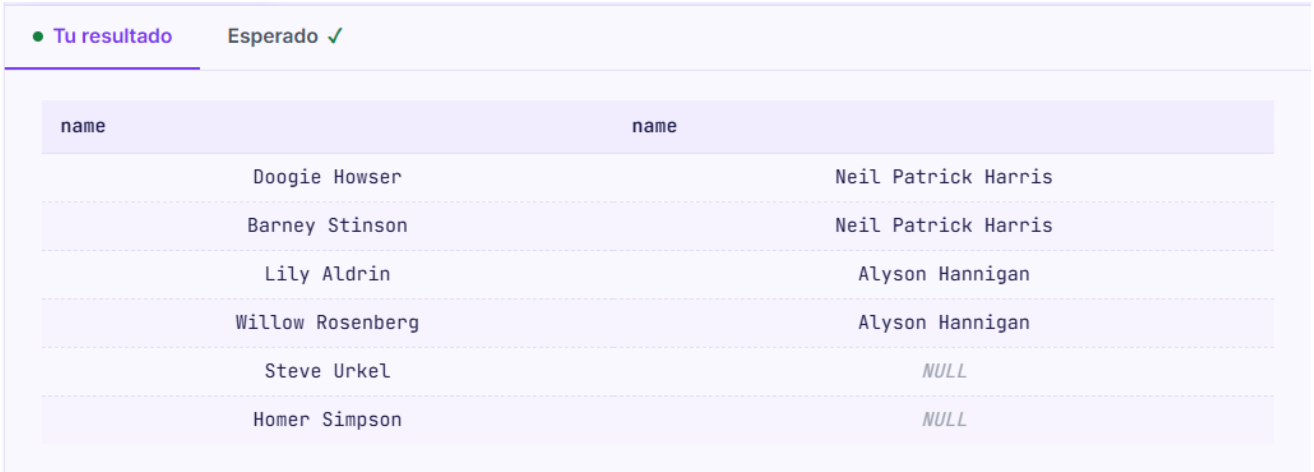


Resultado (Evidencia): La consulta vincula las tablas de personajes y programas de televisión mediante tablas intermedias, filtrando y omitiendo de manera exitosa los registros correspondientes al personaje "Willow Rosenberg" y al programa "How I Met Your Mother", devolviendo únicamente el conjunto de datos esperado.

Aprendí a combinar consultas multitabla avanzadas con filtros restrictivos para acotar conjuntos de datos relacionales. Permite refinar consultas complejas en bases de datos (por ejemplo, excluir usuarios inactivos de un reporte de ventas por región o descartar categorías específicas en auditorías de seguridad) de forma precisa.

Ejercicio 24 – Preservación de registros con LEFT JOIN

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite conservar todos los registros de la tabla principal o "izquierda" (character), asociando los datos pertenecientes a las tablas secundarias cuando existen coincidencias y devolviendo NULL cuando no se encuentra ninguna relación. **Consulta ejecutada:** SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;



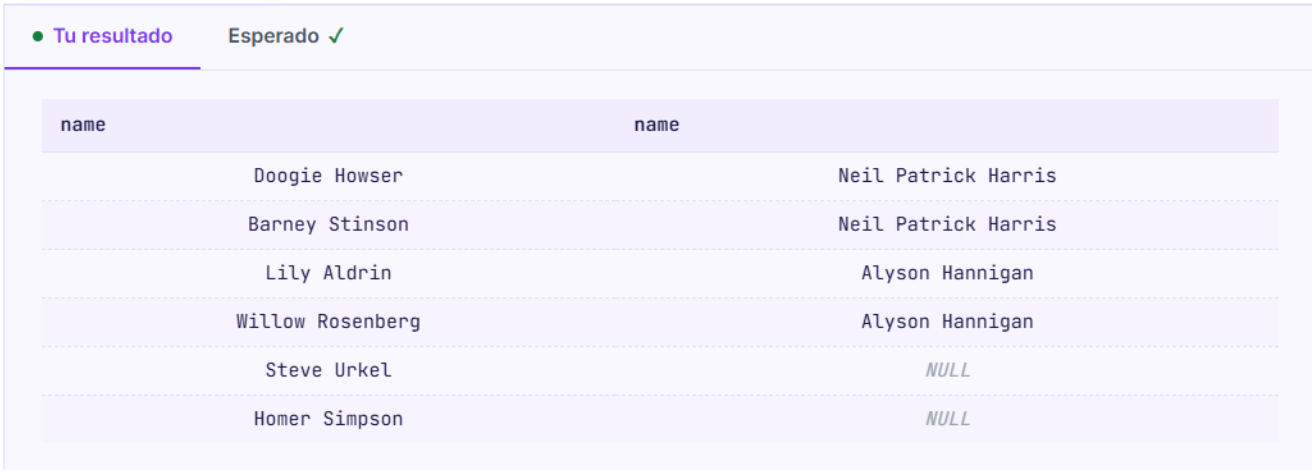
name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	*NULL*
Homer Simpson	*NULL*

Resultado (Evidencia): La consulta vincula exitosamente la tabla character con character_actor y actor mediante LEFT JOIN, garantizando que todos los personajes aparezcan en el resultado final sin omitir a aquellos que carecen de un actor asignado.

Aprendí a utilizar la cláusula LEFT JOIN para realizar consultas donde no se deben perdonar ni perder registros de la tabla de origen a pesar de inconsistencias o ausencias de relaciones en la base de datos. Es fundamental en auditorías para detectar huérfanos, registros incompletos o usuarios que aún no registran ninguna actividad o transacción.

Ejercicio 25 – Simplificación de consultas con Alias de Tabla (AS) y LEFT JOIN

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite asignar renombres temporales o apodos a las tablas mediante la palabra clave AS, simplificando la escritura de referencias de columnas en uniones de múltiples tablas preservando registros mediante LEFT JOIN. **Consulta ejecutada:** SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;



name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	*NULL*
Homer Simpson	*NULL*

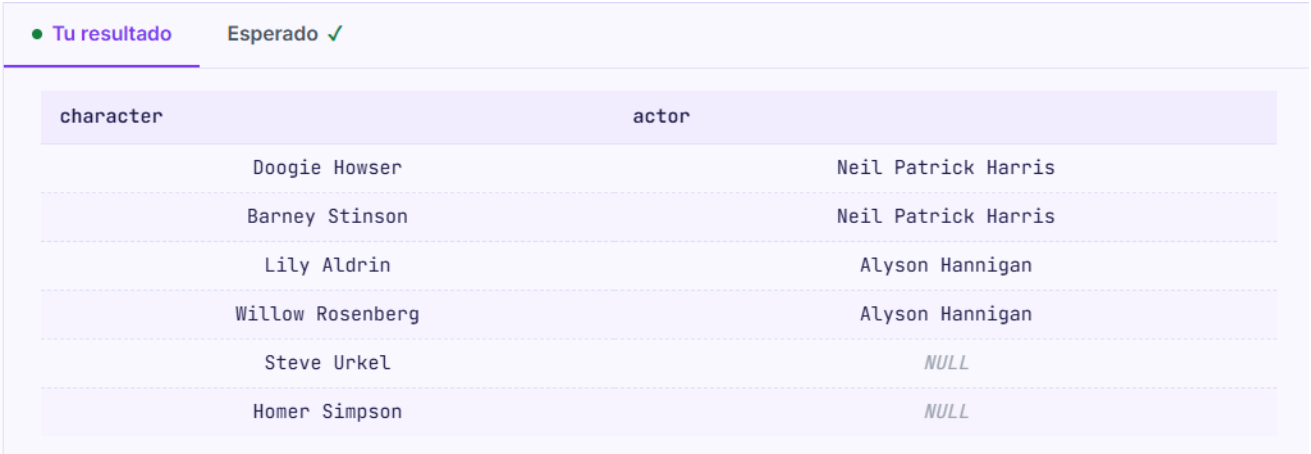
Resultado (Evidencia): La consulta ejecuta un cruce multitabla utilizando los alias c, ca y a, devolviendo el listado completo de personajes junto a sus respectivos actores sin omitir registros y con una sintaxis concisa.

Aprendí a utilizar alias de tablas (AS) combinados con uniones LEFT JOIN para acortar el código SQL y mejorar la legibilidad y mantenibilidad de consultas complejas. Es una práctica estándar en auditorías de bases de datos para escribir scripts limpios sin perder registros de la tabla principal.

Ejercicio 26 – Clarificación de resultados con Alias de Columna (AS)

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite asignar nombres descriptivos específicos a las columnas devueltas en el resultado de una consulta mediante la palabra clave AS, evitando la ambigüedad generada por nombres de columnas idénticos entre tablas relacionadas.

Consulta ejecutada: SELECT c.name AS character, a.name AS actor FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;



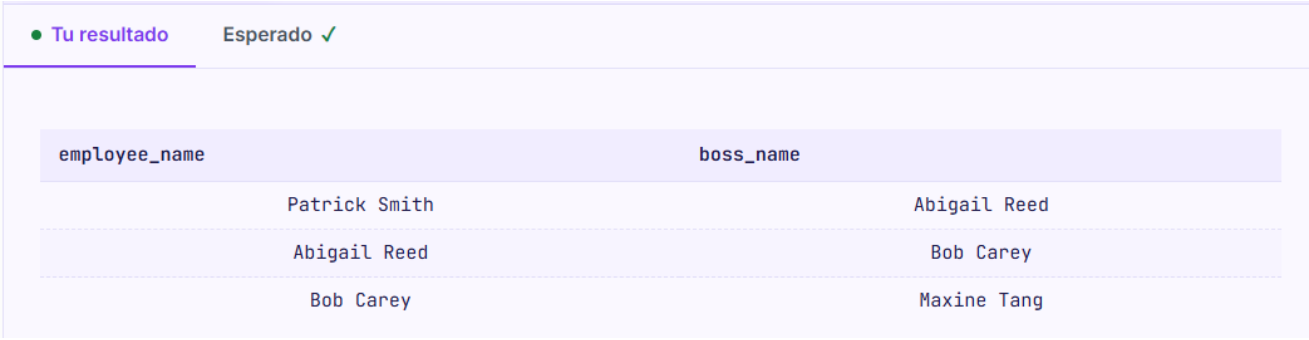
character	actor
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	*NULL*
Homer Simpson	*NULL*

Resultado (Evidencia): La consulta ejecuta uniones izquierdas multitabla utilizando alias tanto para las tablas como para las columnas de salida (character y actor), devolviendo un esquema autodescriptivo limpio y sin duplicidad de cabeceras.

Aprendí a utilizar alias en las columnas para estructurar reportes claros y profesionales en SQL. Es una técnica esencial en auditorías y desarrollo de software para exportar datos con nombres normalizados que eviten confusiones en las capas de aplicación o análisis posteriores.

Ejercicio 27 – Uniones relacionales con la misma tabla (Self Join)

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite relacionar registros de una misma tabla utilizando alias para diferenciar las instancias (por ejemplo, tratar una instancia como "empleado" y la otra como "jefe") a través de una clave foránea que apunta a la misma entidad. **Consulta ejecutada:** SELECT e.name AS employee_name, b.name AS boss_name FROM employees AS e INNER JOIN employees AS b ON e.boss_id = b.id;



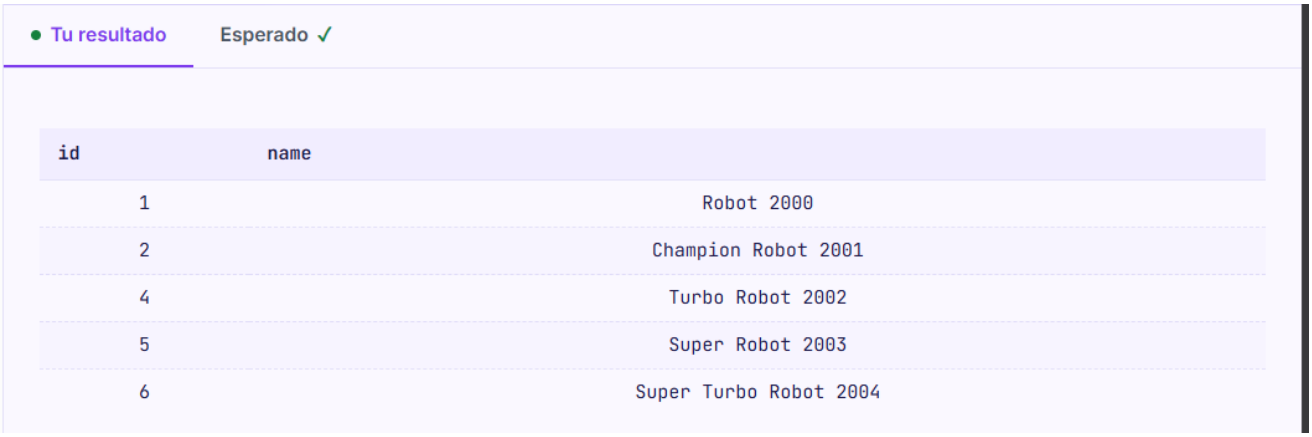
employee_name	boss_name
Patrick Smith	Abigail Reed
Abigail Reed	Bob Carey
Bob Carey	Maxine Tang

Resultado (Evidencia): La consulta ejecuta uniones cruzadas sobre la tabla employees mediante los alias e y b, devolviendo un listado claro que asocia de forma exitosa el nombre de cada empleado con el nombre de su respectivo supervisor.

Aprendí a implementar Self Joins para estructurar consultas sobre jerarquías y relaciones recursivas almacenadas en una única tabla. Es una técnica fundamental en bases de datos relacionales para modelar organigramas empresariales, categorías con subcategorías o redes de referencias directas.

Ejercicio 28 – Filtrado de patrones de texto con LIKE y comodines

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite realizar búsquedas avanzadas de texto flexible utilizando el operador LIKE en combinación con comodines como % (para cero o múltiples caracteres) y _ (para caracteres únicos específicos, ideal para definir rangos de dígitos como los años). **Consulta ejecutada:** SELECT * FROM robots WHERE name LIKE "%Robot 20__%";



id	name
1	Robot 2000
2	Champion Robot 2001
4	Turbo Robot 2002
5	Super Robot 2003
6	Super Turbo Robot 2004

Resultado (Evidencia): La consulta filtra de manera exitosa la tabla robots devolviendo únicamente aquellos registros que contienen la cadena "Robot" seguida de un año válido dentro del siglo XXI (del 2000 al 2099), descartando automáticamente años fuera de rango como el 2111.

Aprendí a utilizar el operador LIKE con comodines posicionales para extraer subconjuntos de datos con patrones de nomenclatura específicos. Es una herramienta indispensable en auditorías de seguridad y análisis forense de bases de datos para localizar anomalías, versiones específicas de software o formatos de datos irregulares en campos de texto libre.

Ejercicio 29 – Sentencias condicionales en consultas con CASE

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite generar lógica condicional dentro de una consulta SQL para transformar o clasificar valores al vuelo según múltiples criterios utilizando bloques CASE, WHEN, THEN, ELSE y END como una nueva columna calculada. **Consulta ejecutada:** SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM friends_of_pickles;



● Tu resultado Esperado ✓

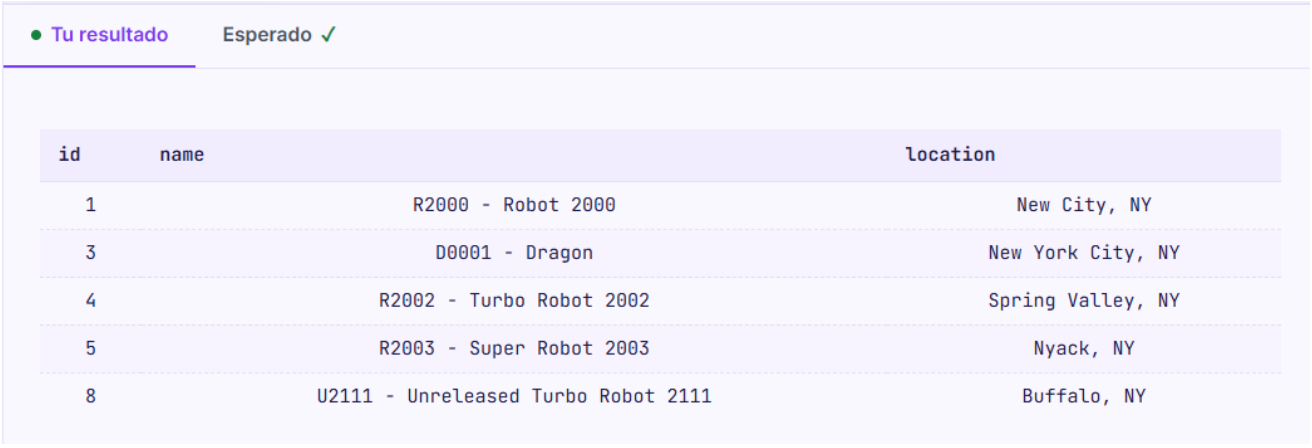
id	name	gender	species	height_cm	sound
1	Dave	male	human	180	talk
2	Mary	female	human	160	talk
3	Fry	male	cat	30	meow
4	Leela	female	cat	25	meow
5	Odie	male	dog	40	bark
6	Jumpy	male	dog	35	bark
7	Sneakers	male	dog	55	bark

Resultado (Evidencia): La consulta evalúa la columna species de la tabla de forma condicional, asignando e imprimiendo dinámicamente la columna sound con el comportamiento de comunicación específico para cada categoría sin alterar los datos originales almacenados.

Aprendí a utilizar declaraciones condicionales CASE en SQL para enriquecer conjuntos de datos directamente en la capa de consulta. Es una técnica fundamental en auditorías y análisis de datos para etiquetar riesgos, categorizar estados o normalizar valores heterogéneos en los reportes generados.

Ejercicio 30 – Extracción de subcadenas con SUBSTR

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite manipular y extraer porciones específicas de cadenas de texto utilizando la función SUBSTR(columna, índice, [longitud]), admitiendo índices negativos para contar posiciones desde el final de la cadena de forma dinámica. **Consulta ejecutada:** SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';



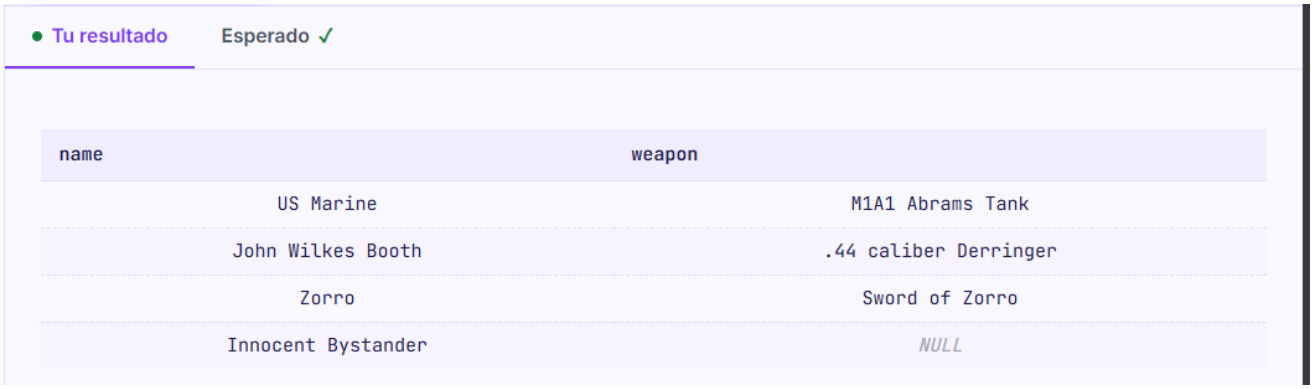
id	name	location
1	R2000 - Robot 2000	New City, NY
3	D0001 - Dragon	New York City, NY
4	R2002 - Turbo Robot 2002	Spring Valley, NY
5	R2003 - Super Robot 2003	Nyack, NY
8	U2111 - Unreleased Turbo Robot 2111	Buffalo, NY

Resultado (Evidencia): La consulta evalúa los registros analizando los últimos caracteres de la columna location mediante un índice negativo en SUBSTR, filtrando y devolviendo de manera exitosa únicamente los robots ubicados en "NY".

Aprendí a utilizar la función SUBSTR para realizar análisis avanzados y extracciones posicionales sobre campos de texto. Es una herramienta clave en auditorías de seguridad y análisis de datos para estandarizar campos, aislar códigos de región o procesar información estructurada en campos libres.

Ejercicio 31 – Gestión de valores nulos con COALESCE

Nivel de Riesgo **Bajo** (es una consulta de lectura avanzada). Permite evaluar una lista ordenada de expresiones o columnas y retornar el primer valor que no sea NULL, facilitando el manejo de datos incompletos o la priorización de recursos jerárquicos. **Consulta ejecutada:** SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;



name	weapon
US Marine	M1A1 Abrams Tank
John Wilkes Booth	.44 caliber Derringer
Zorro	Sword of Zorro
Innocent Bystander	*NULL*

Resultado (Evidencia): La consulta evalúa la jerarquía de armamento para cada luchador en la tabla fighters, asignando con éxito el tank (si está disponible), caso contrario evaluando el gun, y finalmente recurriendo a la sword o devolviendo NULL si no posee ninguna de las tres.

Aprendí a utilizar la función COALESCE para consolidar múltiples campos opcionales en una sola columna con prioridades lógicas. Es una técnica esencial en auditorías de bases de datos, desarrollo de software y análisis financiero para evitar errores por valores nulos y estructurar reportes robustos.

Conclusiones Finales

- **¿Qué aprendí?:** Comprendí a profundidad el funcionamiento de las consultas SQL avanzadas —desde uniones complejas (INNER JOIN, LEFT JOIN, Self Joins) y filtrado de patrones con LIKE hasta el uso de funciones lógicas (CASE, SUBSTR, COALESCE)—. Esto me proporcionó las herramientas analíticas necesarias para estructurar consultas eficientes, identificar la correcta relación entre entidades y detectar anomalías o vulnerabilidades en la manipulación de bases de datos.
- **¿Qué fue lo más desafiante?:** El principal desafío consistió en mantener la coherencia sintáctica al trabajar con nombres de tablas específicos (como diferenciar entre employee y employees) y resolver consultas con estructuras relacionales recursivas o de múltiples condiciones. Las dificultades se superaron mediante la depuración detallada de los errores arrojados por la plataforma y la validación cruzada de las columnas disponibles.
- **¿Qué revisaría a continuación?:** En un entorno real, investigaría más a fondo cómo se conectan las páginas web con estas bases de datos para comprobar que los formularios o campos de texto estén bien protegidos, evitando que un atacante pueda colar comandos maliciosos o ver información que no le corresponde.